گزارش جامع مهندسی معکوس و تحلیل سیستمSmartTask

1. Executive Summary

SmartTaskیک سامانه مدیریت پروژه چابک (Agile Project Management) با رابط کاربری فارسی و پشتیبانی RTLاست. این سامانه بر پایه ASP.NET Core 8.0 MVC و SQL Server ساخته شده و شامل سلسله‌مراتب سازمانی **Workspace → Project → Sprint → UserStory → Task**می‌باشد. ویژگی‌های هوشمند آن شامل **موتور اولویت‌بندی خودکار(Priority Engine) ، تحلیل ریسک تأخیر(Delay Risk Analysis) ، سلامت پروژه (Project Health Score)، تحلیل حجم کاری(Workload Analysis) ، کاسکاد خودکار موعد وابستگی‌ها (Overdue Cascade)، گزارش‌سازی هوشمند اسپرینت با AI** و **شکستن خودکار تسک‌ها با AI** می‌شود.

---

2. Project Identity

Field	Value
Name	SmartTask (SmartTask.Web)
Type	ASP.NET Core 8.0 MVC Web Application
Domain	Agile Project Management
Language	Persian/Farsi (RTL-first)
Architecture	MVC + Service Layer + Repository + Unit of Work
Database	SQL Server (via EF Core 8.0.28)
Target	University thesis project

---

3. Problem Domain

3.1 Problem Statement

🟢 Verified

تیم‌های نرم‌افزاری برای مدیریت پروژه‌های چابک نیاز به سامانه‌ای دارند که:

- اولویت‌بندی هوشمند تسک‌ها بر اساس فوریت زمانی، تأثیر وابستگی و حجم کاری اعضاء انجام دهد
- ریسک تأخیر در پروژه را به‌صورت کمّی محاسبه و پیش‌بینی کند
- سلامت کلی پروژه را با فرمول ترکیبی ارزیابی کند
- زنجیره وابستگی تسک‌ها را ردیابی و کاسکاد تأخیر را خودکار مدیریت کند

- ارتباطات تیمی درون پروژه را با چت بلادرنگ فراهم کند
- گزارش‌های هوشمند پایان اسپرینت را با AI تولید کند

3.2 Target Users

🟢 Verified

- تیم‌های توسعه نرم‌افزار
- مدیران پروژه(Project Managers)
- توسعه‌دهندگان(Developers)
- تسترها(Testers)
- مدیران سیستم(Admin)

---

4. Target Users (Actors)

Identity Roles

🟢 Verified — Infrastructure/Seed/RoleSeeder.cs

- **Admin**: دسترسی کامل به داشبورد مدیریتی
- **ProjectManager**: مدیریت پروژه‌ها
- **Member**: عضو عادی

Workspace Roles

🟢 Verified — Models/Enums/WorkspaceRoleType.cs

- **Owner**(مالک): مالکیت Workspace
- **Admin**(مدیر): مدیریت کلی
- **ProjectManager**(مدیر پروژه): مدیریت پروژه‌ها
- **Developer** (توسعه‌دهنده)
- **Tester** (تستر)
- **Viewer** (مشاهده‌گر)

Project Roles

🟢 Verified — Models/Enums/ProjectRoleType.cs

- **Owner** (مالک پروژه)
- **Manager** (مدیر پروژه)
- **Developer** (توسعه‌دهنده)
- **Tester** (تستر)
- **Guest** (مهمان)

Team Roles

🟢 Verified — Models/Enums/TeamRoleType.cs

- **Leader** (رهبر تیم)
- **Member** (عضو تیم)
- **Guest** (مهمان)

---

5. Core Objectives

🟢 Verified — based on implemented features

- **مدیریت سلسله‌مراتبی**: Workspace → Project → Sprint → Story → Task
- **اولویت‌بندی هوشمند**: موتور امتیازدهی چندعامله
- **تحلیل ریسک**: محاسبه کمّی ریسک تأخیر پروژه
- **سلامت پروژه**: امتیاز ترکیبی چند بعدی
- **کاسکاد خودکار**: به‌روزرسانی خودکار موعد وابسته‌ها
- **ارتباطات بلادرنگ**: چت گروهی با SignalR
- **گزارش‌سازی هوشمند**: تولید گزارش با AI
- **تحلیل حجم کاری**: نظارت بر بار کاری اعضاء

---

6. Architecture

6.1 Architectural Pattern

🟢 Verified — Evidence from Program.cs, Services/, Infrastructure/

MVC + Service Layer + Generic Repository + Unit of Work

6.2 Data Flow

User (Browser)

↓

Razor View / JavaScript (AJAX)

↓

ASP.NET Core MVC Controller

↓

Service Interface → Service Implementation

↓

ApplicationDbContext (EF Core)

↓

SQL Server

6.3 Layer Responsibilities

Layer	Location	Responsibility
Presentation	Views/, wwwroot/js/, wwwroot/css/	UI rendering, client-side logic
Controller	Controllers/	Request handling, routing, view model mapping
Service	Services/Implementations/	Business logic, validation, orchestration
Repository	Infrastructure/Repositories/	Generic data access (IGenericRepository)
Data	Data/Context/, Data/Configurations/	EF Core DbContext, entity configuration
Background	Infrastructure/BackgroundJobs/	Scheduled tasks (reminder, cascade)
Realtime	Hubs/	SignalR hubs for notifications & chat

Infrastructure	Infrastructure/Services/	Current user, presence tracking

---

7. Technology Stack

7.1 Backend

🟢 Verified — SmartTask.Web.csproj

Package	Version	Purpose
Microsoft.NET.Sdk.Web	net8.0	ASP.NET Core 8.0
Microsoft.EntityFrameworkCore.SqlServer	8.0.28	SQL Server provider
Microsoft.AspNetCore.Identity.EntityFrameworkCore	8.0.8	Identity & auth
Microsoft.AspNetCore.Authentication.Google	8.0.29	Google OAuth (commented out in Program.cs)
Microsoft.EntityFrameworkCore.InMemory	8.0.28	Testing
ClosedXML	0.105.1	Excel export
QuestPDF	2026.7.2	PDF generation
xunit	2.9.3	Unit testing
Moq	4.20.72	Mocking

7.2 Frontend

🟢 Verified — wwwroot/lib/, wwwroot/css/, wwwroot/js/

- Bootstrap (via libman)
- Font Awesome (icons)
- Vazirmatn font (Persian typography)
- Chart.js (dashboards)
- SweetAlert2 (confirmations)
- SignalR JS client

- Custom CSS (41 files)
- Custom JS (33 files)

7.3 Realtime

🟢 Verified — Hubs/NotificationHub.cs, Hubs/ChatHub.cs

- **SignalR** for notification delivery and group chat

7.4 AI Integration

🟢 Verified — Services/AI/AiClientService.cs

- **LM Studio** (local LLM via HTTP API) — model: google/gemma-4-12b
- Used for: Sprint report generation, Task breakdown, Risk narrative

7.5 Push Notifications

🟢 Verified — Services/Implementations/WebpushrService.cs

- **Webpushr** API for browser push notifications (chat messages)

7.6 Database

🟢 Verified — appsettings.json

- **SQL Server** (remote instance)

7.7 Email

🟢 Verified — Services/Email/

- **SMTP Gmail** for email verification, password reset, invitations

---

8. Project Structure

```
SmartTask.Web/
├── Controllers/          (41 controllers)
├── Services/
│   ├── Implementations/  (46 services)
│   ├── Interfaces/       (46 interfaces)
```

```
│   ├── AI/              (AiClientService, OpenAiSettings)
│   ├── Email/           (EmailService)
│   └── Files/           (FileUploadService)
├── Models/
│   ├── Entities/        (34 entities)
│   ├── Enums/           (23 enums)
│   ├── ViewModels/      (30+ subdirectories)
│   └── DTOs/            (2 DTOs)
├── Data/
│   ├── Context/         (ApplicationDbContext)
│   ├── Configurations/  (34 EF configurations)
│   └── Migrations/      (20 migrations + snapshot)
├── Infrastructure/
│   ├── BackgroundJobs/  (2 background services)
│   ├── Repositories/    (GenericRepository + UnitOfWork)
│   ├── Seed/            (RoleSeeder + AdminSeeder)
│   ├── Services/        (CurrentUserService, PresenceTracker)
│   └── Interfaces/      (IGenericRepository, IUnitOfWork, etc.)
├── Hubs/                (NotificationHub, ChatHub)
├── ViewComponents/      (HeaderViewComponent, NotificationBellViewComponent)
├── Views/               (30 view directories + Shared)
├── Common/              (Extensions, Filters, Helpers, Constants)
├── Middlewares/         (excluded from build)
├── Repositories/        (excluded from build)
├── Components/          (SmartTaskSimple.jsx - React component)
```

```
└── wwwroot/
    ├── css/          (41 CSS files)
    ├── js/           (33 JS files)
    ├── lib/          (Bootstrap, jQuery, etc.)
    └── fonts/        (Vazirmatn)
```

---

9. Complete Entity Inventory

BaseEntity

🟢 Verified — Models/Entities/BaseEntity.cs

Field	Type	Purpose
Id	int	Primary key
CreatedBy	string?	Creator identifier
CreatedDate	DateTime	Creation timestamp
ChangeUser	string?	Last modifier
ChangeDate	DateTime?	Last modification
ViewState	bool	Soft delete flag

ApplicationUser

🟢 Verified — Models/Entities/ApplicationUser.cs

Field	Type	Default	Purpose
FirstName	string	required	First name
LastName	string	required	Last name
Avatar	string?	null	Profile photo
Bio	string?	null	Biography
JobTitle	string?	null	Job title

IsActive	bool	true	Active flag
LastLoginDate	DateTime?	null	Last login
Theme	ThemeType	Light	UI theme
TimeZone	string	Asia/Tehran	Timezone
DateFormat	DateFormatType	Jalali	Date format
ShowFullName	bool	true	Display name
TaskDensity	TaskDensityType	Comfortable	UI density
AutoCascadeDependencyDates	bool	true	Auto cascade toggle
DefaultWorkspaceId	int?	null	Default workspace
WebpushrSubscriberId	long?	null	Push subscriber
UserSessions	ICollection<UserSession>		Device tracking

Workspace

🟢 Verified — Models/Entities/Workspace.cs

Field	Type	Purpose
Name	string	Workspace name
Description	string?	Description
Logo	string?	Logo path
Color	string?	Brand color
TimeZone	string?	Timezone
Visibility	VisibilityType	Private/Public
IsActive	bool	Active flag
OwnerId	int FK → ApplicationUser	

Projects	ICollection<Project>	Child projects
Teams	ICollection<Team>	Child teams
Members	ICollection<WorkspaceMember>	Members

Project

Verified — Models/Entities/Project.cs

Field	Type	Purpose
WorkspaceId	int	FK → Workspace
Name	string	Project name
Key	string	Short code (e.g., "ST")
Description	string?	Description
Color	string?	Brand color
Icon	string?	FontAwesome icon
StartDate	DateTime?	Start date
DueDate	DateTime?	Due date
EndDate	DateTime?	Actual end
Status	ProjectStatusType	Planning/Active/OnHold/Completed/Cancelled
Priority	ProjectPriorityType	Low/Medium/High/Critical
IsArchived	bool	Archive flag
Members	ICollection<ProjectMember>	Members
Sprints	ICollection<Sprint>	Sprints
UserStories	ICollection<UserStory>	Stories
Backlog	Backlog?	Product Backlog

Labels	ICollection<Label>	Labels
ProjectTeams	ICollection<ProjectTeam>	Team assignments

Sprint

🟢 Verified — Models/Entities/Sprint.cs

Field	Type	Purpose
ProjectId	int	FK → Project
Name	string	Sprint name
Goal	string?	Sprint goal
StartDate	DateTime	Start
EndDate	DateTime	End
Capacity	int	Story points capacity
Status	SprintStatusType	Planning/Active/Completed/Cancelled
UserStories	ICollection<UserStory>	Assigned stories

UserStory

🟢 Verified — Models/Entities/UserStory.cs

Field	Type	Purpose
ProjectId	int	FK → Project
BacklogId	int	FK → Backlog
SprintId	int?	FK → Sprint (nullable)
OwnerId	int?	FK → ApplicationUser
Title	string	Story title
Description	string?	Description

AcceptanceCriteria	string?	Acceptance criteria
StoryPoint	int	Effort estimate
BusinessValue	int	Business value
Order	int	Sort order in backlog
Priority	StoryPriorityType	Priority level
Status	StoryStatusType	New/Ready/InProgress/Done/Closed
Tasks	ICollection<TaskItem>	Child tasks

TaskItem

🟢 Verified — Models/Entities/TaskItem.cs

Field	Type	Purpose
UserStoryId	int	FK → UserStory
Title	string	Task title
Description	string?	Description
Status	TaskStatusType	ToDo/InProgress/InReview/Done/Cancelled
Priority	TaskPriorityType	Lowest/Low/Medium/High/Highest
Type	TaskType	Task/Bug/Feature/Improvement/TechnicalDebt
Estimate	int	Hours estimate
StartDate	DateTime?	Start
DueDate	DateTime?	Due date
CompletedDate	DateTime?	Completion
SubTasks	ICollection<SubTaskItem>	Sub-tasks

Assignments	ICollection<TaskAssignment>	Assignees
Comments	ICollection<Comment>	Comments
Attachments	ICollection<Attachment>	Files
Checklists	ICollection<Checklist>	Checklists
TaskLabels	ICollection<TaskLabel>	Labels
Reminders	ICollection<Reminder>	Reminders
ActivityLogs	ICollection<ActivityLog>	Activity
TimeLogs	ICollection<TimeLog>	Time tracking

TaskDependency

🟢 Verified — Models/Entities/TaskDependency.cs

Field	Type	Purpose
TaskItemId	int	FK → TaskItem (dependent)
DependsOnTaskItemId	int	FK → TaskItem (prerequisite)
IsRequired	bool	Required/optional dependency

OverdueCascadeLog

🟢 Verified — Models/Entities/OverdueCascadeLog.cs

Field	Type	Purpose
SourceTaskId	int	FK → Source task
ImpactedTaskId	int	FK → Impacted task
DelayDaysApplied	int	Days delayed
AppliedDate	DateTime	When applied

ChatMessage

🟢 Verified — Models/Entities/ChatMessage.cs

Field	Type	Purpose
ProjectId	int	FK → Project
SenderId	int	FK → ApplicationUser
Content	string	Message text
Type	ChatMessageType	Text/Image/File/System
AttachmentPath	string?	File path
ReplyToMessageId	int?	Reply reference
IsEdited	bool	Edit flag
IsPinned	bool	Pin flag
Replies	ICollection<ChatMessage>	Thread replies

ChatMessageReaction

🟢 Verified — Models/Entities/ChatMessageReaction.cs

Field	Type	Purpose
ChatMessageId	int	FK → ChatMessage
UserId	int	FK → ApplicationUser
Emoji	string	Reaction emoji

ChatReadState

🟢 Verified — Models/Entities/ChatReadState.cs

Field	Type	Purpose
ProjectId	int	FK → Project
ApplicationUserId	int	FK → ApplicationUser
LastReadMessageId	int	Last read message
LastReadDate	DateTime	Timestamp

Notification

Verified — Models/Entities/Notification.cs

Field	Type	Purpose
ApplicationUserId	int	FK → User
Title	string	Title
Message	string	Body
Type	NotificationType	8 types
IsRead	bool	Read status
ReadDate	DateTime?	When read

UserNotificationPreference

Verified — Models/Entities/UserNotificationPreference.cs

Field	Type	Purpose
ApplicationUserId	int	FK → User
NotificationType	NotificationType	Type
IsEnabled	bool	Preference

Other Entities

Verified

- **Team**: Workspace team (Name, Color, Logo, IsPrivate, IsArchived)
- **TeamMember**: Team membership with TeamRoleType
- **ProjectTeam**: Many-to-many Project ↔ Team
- **ProjectMember**: Project membership with WeeklyCapacityHours
- **WorkspaceMember**: Workspace membership with WorkspaceRoleType
- **WorkspaceInvitation**: Invitation with Token, Status, ExpiryDate
- **Label**: Project labels with name and color

- **TaskLabel**: Many-to-many Task ↔ Label
- **Comment**: Task comments
- **Attachment**: Task file attachments
- **Checklist/ChecklistItem**: Task checklists
- **SubTaskItem**: Task sub-tasks
- **TaskAssignment**: Task ↔ User assignment
- **Reminder**: Task reminders with auto-deadline detection
- **ActivityLog**: User activity tracking
- **TimeLog**: Time tracking per task
- **OffroadTask**: Out-of-scope tasks
- **TaskTradeRequest**: Task exchange between members
- **SprintReport**: AI-generated sprint reports
- **UserSession**: Device/session tracking
- **Backlog**: Product backlog container

---

10. Database Architecture

10.1 Migration History

🟢 Verified — Migrations/ (20 migrations)

1. InitialIdentity — Identity tables + core entities
2. AddWorkspaceInvitations — Invitation system
3. AddWorkspaceTimeZone — Workspace timezone
4. UpdateWorkspace — Workspace updates
5. ProjectTeamManyToMany — Project-Team M
6. AddSprintFeature — Sprint, UserStory, TaskItem
7. UpdateUserStoryForBacklog — Backlog integration
8. AddUserStoryOwner — Story owner

9. AddUserStoryOwnerField — Owner field
10. AddApplicationUserCreatedDate — User created date
11. AddOffroadTask — Offroad tasks
12. AddProjectMemberWeeklyCapacity — Capacity hours
13. AddOverdueCascadeLog — Cascade tracking
14. AddSprintReport — Sprint reports
15. AddTaskTradeRequest — Task trading
16. AddUserSettingsAndNotificationPreferences — Settings & preferences
17. MakeProjectKeyIndexFiltered — Filtered index
18. MakeTeamAndSprintIndexesFiltered — Filtered indexes
19. AddProjectChat — Chat entities
20. AddWebpushrSubscriberId — Push notification subscriber
21. AddAutoCascadeDependencyDatesSetting — Auto cascade toggle
22. RemoveLanguageSetting — Language setting removal
23. AddChatReactionsAndPin — Chat reactions & pins
24. AddUserSessionTracking — Device sessions

10.2 Tables Summary

Table	Purpose	Key Relationships
AspNetUsers	Identity users	Parent of all user relationships
Workspaces	Workspace container	OwnerId → Users
WorkspaceMembers	Workspace membership	WorkspaceId, ApplicationUserId (unique)
WorkspaceInvitations	Invitations	WorkspaceId → Workspaces
Projects	Project container	WorkspaceId → Workspaces

ProjectMembers	Project membership	ProjectId, ApplicationUserId (unique), WeeklyCapacityHours
ProjectTeams	Project-Team M	ProjectId, TeamId
Teams	Teams	WorkspaceId → Workspaces
TeamMembers	Team membership	TeamId, ApplicationUserId
Sprints	Sprint periods	ProjectId → Projects
Backlogs	Product backlogs	ProjectId → Projects (1:1)
UserStories	Stories	ProjectId, BacklogId, SprintId?, OwnerId?
TaskItems	Tasks	UserStoryId → UserStories
SubTaskItems	Sub-tasks	TaskItemId → TaskItems
TaskAssignments	Task-user	TaskItemId, ApplicationUserId
TaskDependencies	Dependencies	TaskItemId, DependsOnTaskItemId (unique, NoAction)
TaskLabels	Task-label M	TaskItemId, LabelId
Labels	Project labels	ProjectId → Projects
Comments	Task comments	TaskItemId, ApplicationUserId
Attachments	File attachments	TaskItemId, ApplicationUserId
Checklists	Checklists	TaskItemId → TaskItems
ChecklistItems	Checklist items	ChecklistId → Checklists
Reminders	Reminders	TaskItemId, ApplicationUserId
Notifications	System notifications	ApplicationUserId
UserNotificationPreferences	Preferences	ApplicationUserId, NotificationType

ActivityLogs	Activity tracking	ApplicationUserId, TaskItemId?
TimeLogs	Time tracking	TaskItemId, ApplicationUserId
OffroadTasks	Out-of-scope tasks	ProjectId
TaskTradeRequests	Task exchange	ProjectId, RequesterUserId, TargetUserId, RequesterTaskId, TargetTaskId?
SprintReports	AI reports	SprintId, GeneratedByUserId
OverdueCascadeLogs	Cascade tracking	SourceTaskId, ImpactedTaskId
ChatMessages	Chat messages	ProjectId, SenderId, ReplyToMessageId?
ChatMessageReactions	Reactions	ChatMessageId, UserId
ChatReadStates	Read tracking	ProjectId, ApplicationUserId
UserSessions	Device sessions	ApplicationUserId

---

11. ERD (Entity Relationship Diagram)

Key Relationships

🟢 Verified — Data/Configurations/

ApplicationUser

├— 1:N WorkspaceMemberships → WorkspaceMember → Workspace

├— 1:N ProjectMemberships → ProjectMember → Project

├— 1:N TeamMemberships → TeamMember → Team

├— 1:N TaskAssignments → TaskAssignment → TaskItem

├— 1:N Comments → Comment → TaskItem

├— 1:N Attachments → Attachment → TaskItem

├— 1:N Notifications → Notification

├— 1:N ActivityLogs → ActivityLog

├── 1:N TimeLogs → TimeLog → TaskItem

├── 1:N Reminders → Reminder → TaskItem

├── 1:N NotificationPreferences → UserNotificationPreference

└── 1:N UserSessions → UserSession

Workspace

├── N:1 Owner → ApplicationUser

├── 1:N Members → WorkspaceMember

├── 1:N Projects → Project

├── 1:N Teams → Team

└── 1:N Invitations → WorkspaceInvitation

Project

├── N:1 Workspace → Workspace

├── 1:N Members → ProjectMember

├── 1:N Sprints → Sprint

├── 1:1 Backlog → Backlog

├── 1:N UserStories → UserStory

├── 1:N Labels → Label

├── 1:N ProjectTeams → ProjectTeam → Team

└── 1:N ChatMessages → ChatMessage

Sprint

├── N:1 Project → Project

└── 1:N UserStories → UserStory

UserStory

├— N:1 Project → Project

├— N:1 Backlog → Backlog

├— N:1 Sprint? → Sprint

├— N:1 Owner? → ApplicationUser

└— 1:N Tasks → TaskItem

TaskItem

├— N:1 UserStory → UserStory

├— 1:N SubTasks → SubTaskItem

├— 1:N Assignments → TaskAssignment

├— 1:N Comments → Comment

├— 1:N Attachments → Attachment

├— 1:N Checklists → Checklist → ChecklistItems

├— 1:N TaskLabels → TaskLabel → Label

├— 1:N Reminders → Reminder

├— 1:N ActivityLogs → ActivityLog

├— 1:N TimeLogs → TimeLog

├— M:N Dependencies (TaskDependency, self-referential)

└— 1:N OverdueCascadeLogs (as source/impacted)

ChatMessage

├— N:1 Project → Project

├— N:1 Sender → ApplicationUser

├── N:1 ReplyTo? → ChatMessage (self-referential)

├── 1:N Replies → ChatMessage

└── 1:N Reactions → ChatMessageReaction → ApplicationUser

Circular Dependencies & Cascade Concerns

🟡 Inferred

- **TaskDependency** uses DeleteBehavior.NoAction on both FKs to avoid multiple cascade paths (both FKs point to TaskItems). This is correctly handled.
- **ProjectMember** uses DeleteBehavior.Restrict on both FKs — prevents accidental cascade deletes.

---

12. Controllers (41 Total)

AccountController

🟢 Verified — Controllers/AccountController.cs

Action	HTTP	Purpose
Register	GET/POST	User registration with email confirmation
Login	GET/POST	Password login with session tracking
ExternalLogin	POST	Google OAuth initiation
ExternalLoginCallback	GET	Google OAuth callback
Logout	POST	Sign out + session revocation
ForgotPassword	GET/POST	Password reset email
ResetPassword	GET/POST	Password reset
ConfirmEmail	GET	Email confirmation
Profile	GET/POST	Profile editing with avatar upload
RemoveAvatar	POST	Avatar removal

ChangePassword	GET/POST	Password change
SaveWebpushrSubscriber	POST	Push notification subscriber registration
TestEmail	GET	Email testing

HomeController

Verified

Action	HTTP	Purpose
Index	GET	User dashboard (personal overview)

WorkspaceController

Verified

Action	HTTP	Purpose
Index	GET	List workspaces
Details	GET	Workspace dashboard
Create	GET/POST	Create workspace
Edit	GET/POST	Edit workspace
Settings	GET/POST	Workspace settings

ProjectController

Verified

Action	HTTP	Purpose
Create	GET/POST	Create project
Details	GET	Project details
Edit	GET/POST	Edit project
Archive/Restore	POST	Archive management

SprintController, BacklogController, TaskController, TaskBoardController

🟢 Verified — full CRUD for sprint lifecycle, backlog management, task operations, board view

DependencyController, TaskDependencyController

🟢 Verified — dependency graph, risk overview, add/remove dependencies

WorkloadController

🟢 Verified — workload analysis per project/sprint

DelayRiskController

🟢 Verified — risk overview and AI narrative

ChatController

🟢 Verified — chat UI, AJAX endpoints for messages, search, members

ReportController, ProjectReportController, WorkspaceReportController, SprintReportController

🟢 Verified — reports with PDF/Excel export and AI sprint reports

SettingsController

🟢 Verified — Account, Notifications, Appearance, Workspace, Management settings, Security (sessions, logout all, delete account)

NotificationController

🟢 Verified — notification CRUD, mark read, SignalR integration

AdminController

🟢 Verified — admin dashboard

Other Controllers

🟢 Verified

- **TeamController**: Team CRUD
- **ProjectMemberController**: Project member management
- **WorkspaceMemberController**: Workspace member management with invite modal

- **ProjectTeamController**: Project-team assignment
- **LabelController**: Label management
- **CommentController**: Task comments
- **AttachmentController**: File uploads
- **ChecklistController**: Checklist management
- **SubTaskController**: Sub-task management
- **TaskAssignmentController**: Task assignment
- **TaskLabelController**: Task-label association
- **TimeLogController**: Time tracking
- **ReminderController**: Reminder management
- **OffroadController**: Offroad tasks
- **TaskTradeController**: Task exchange
- **UserStoryController**: User story management
- **ActivityController**: Activity logs
- **Searchcontroller**: Global search
- **AgileDashboardController**: Agile metrics dashboard

---

13. Services (46 Services)

Core Business Logic Services

1. PriorityEngineService ⭐ (Algorithm)

🟢 Verified — Services/Implementations/PriorityEngineService.cs
Purpose: Smart priority suggestion for tasks based on multi-factor scoring.

Algorithm:

TotalScore = UrgencyScore + DependencyScore + WorkloadScore (0-100)

UrgencyScore (0-40):

- No due date: 10

- Overdue: 40

- Today: 40

- Days until due: 40 - (daysUntilDue × 40/30)

DependencyScore (0-35):

- min|requiredChainCount × 7, 35

WorkloadScore (0-25):

- Utilization > 100%: 25

- Utilization >= 80%: 15

Priority Mapping:

≤ 20 → Lowest

≤ 40 → Low

≤ 60 → Medium

≤ 80 → High

> 80 → Highest

Source: Services/Implementations/PriorityEngineService.cs

2. DelayRiskService ⭐ (Algorithm)

🟢 Verified — Services/Implementations/DelayRiskService.cs
Purpose: Quantitative delay risk analysis for projects.

Algorithm:

RiskScore = OverdueScore + WorkloadScore + DependencyScore + CascadeScore (0-100)

OverdueScore (0-40):

overdueRatio × 40

WorkloadScore (0-30):

overloadRatio × 30

DependencyScore (0-20):

min|riskyChainCount × 4, 20

CascadeScore (0-10):

min|recentCascadeCount × 2, 10

Risk Level:

≤ 25 → low

≤ 50 → medium

≤ 75 → high

> 75 → critical

Source: Services/Implementations/DelayRiskService.cs

3. ProjectHealthService ⭐ (Algorithm)

🟢 Verified — Services/Implementations/ProjectHealthService.cs
Purpose: Composite project health scoring.

Algorithm:

HealthScore = ScheduleHealth × 0.30 + WorkloadHealth × 0.25 + DependencyHealth × 0.20 + DeliveryHealth × 0.25

ScheduleHealth = 100 - (overdueRatio × 100)

WorkloadHealth = 100 - (overloadRatio × 100)

DependencyHealth = max(0, 100 - min(riskyChainCount × 10, 100))

DeliveryHealth = (completedTasks / totalTasks) × 100

Health Level:

excellent ← 85 ≤ (عالی)

good ← 70 ≤ (خوب)

fair ← 50 ≤ (نیازمند توجه)

poor ← 50 > (بحرانی)

Source: Services/Implementations/ProjectHealthService.cs

4. TaskDependencyService

🟢 Verified — Services/Implementations/TaskDependencyService.cs
Purpose: Dependency management, cycle detection, impact analysis, risk graph.

- **WouldCreateCycleAsync**: BFS cycle detection before adding dependency
- **GetImpactedTasksAsync**: BFS traversal of dependency chain
- **GetProjectRiskOverviewAsync**: Identifies risky dependency chains
- **GetDependencyGraphAsync**: Full graph for visualization
- **GetCascadeInfoAsync**: History of cascade operations

5. WorkloadAnalysisService

🟢 Verified — Services/Implementations/WorkloadAnalysisService.cs
Purpose: Per-member workload analysis with capacity management.

- Calculates assigned hours per member (split across assignees)
- Status levels: under (<80%), balanced (80-100%), overloaded (>100%)
- Supports both project-level and sprint-level analysis

6. TaskBreakdownService ⭐ (AI)

🟢 Verified — Services/Implementations/TaskBreakdownService.cs
Purpose: AI-powered task decomposition into sub-tasks.

- Sends task title + description to LLM

- Returns 3-7 Persian sub-task titles

7. SprintReportAiService ⭐ (AI)

🟢 Verified — Services/Implementations/SprintReportAiService.cs
Purpose: AI-generated sprint retrospective reports.

- Collects sprint stats (story points, completion rate, top contributors)
- Sends structured data to LLM with scrum master persona prompt
- Persists generated reports

8. OverdueCascadeBackgroundService ⭐ (Background Job)

🟢 Verified — Infrastructure/BackgroundJobs/OverdueCascadeBackgroundService.cs
Purpose: Automatic cascade of due dates when upstream tasks are overdue.

- Runs every 30 minutes
- Checks AutoCascadeDependencyDates user setting
- Logs cascades to prevent duplicates
- Sends notifications to impacted assignees

9. ReminderBackgroundService

🟢 Verified — Infrastructure/BackgroundJobs/ReminderBackgroundService.cs
Purpose: Automatic deadline reminders.

- Runs every 15 minutes
- 24-hour and 1-hour deadline notifications
- Manual reminder processing

10. NotificationService

🟢 Verified — Services/Implementations/NotificationService.cs
Purpose: Notification creation, delivery, and management.

- Creates notifications in database
- Sends real-time via SignalR NotificationHub
- Respects UserNotificationPreference
- Batch operations for efficiency

- ExecuteUpdateAsync for performance

11. ChatService

🟢 Verified — Services/Implementations/ChatService.cs
Purpose: Full-featured group chat per project.

- Messages with threads (reply), edit, delete
- Reactions (emoji), pin/unpin
- Read state tracking (ChatReadState)
- Mention parsing with regex
- Rate limiting (5 messages per 3 seconds)
- Online/offline presence via PresenceTracker
- Search with pagination

12. ReportExportService

🟢 Verified — Services/Implementations/ReportExportService.cs
Purpose: PDF and Excel report generation.

- QuestPDF for PDF generation
- ClosedXML for Excel generation
- Workspace and Project report variants

13. WebpushrService

🟢 Verified — Services/Implementations/WebpushrService.cs
Purpose: Browser push notifications for chat messages.

14. UserSessionTracker

🟢 Verified — Services/Implementations/UserSessionTracker.cs
Purpose: Device/session tracking for security.

- Tracks login with User-Agent parsing (browser + OS detection)
- Manages active sessions
- Revokes sessions (all or individual)

15-46. Other Services

🟢 Verified — Standard CRUD operations

- **BaseService<T>**: Generic CRUD with Repository + UnitOfWork
- **WorkspaceService, ProjectService, SprintService, TaskService**: Entity management
- **WorkspaceMemberService, ProjectMemberService, TeamMemberService**: Membership
- **WorkspaceInvitationService**: Invitation with token-based acceptance
- **WorkspaceDashboardService, ProjectDashboardService, UserDashboardService, AdminDashboardService**: Dashboard data
- **WorkspaceReportService, ProjectReportService**: Report data aggregation
- **CommentService, AttachmentService, ChecklistService**: Task collaboration
- **LabelService, TaskLabelService**: Label management
- **TimeLogService, ActivityLogService**: Tracking
- **SettingsService, DateFormatService**: User preferences
- **CurrentUserService, CurrentContextService**: Request context
- **PresenceTracker** (Singleton): Online user tracking
- **BacklogService, UserStoryService, SubTaskService, TaskAssignmentService, TaskTradeService, OffroadTaskService**: Feature-specific

---

14. Business Logic Rules

Rule 1: Workspace Hierarchy

🟢 Verified

کاربر عضو Workspace ← می‌تواند Workspace را ببیند

کاربر عضو Project ← عضو Workspace مربوطه محسوب می‌شود

مالک Workspace ← به همه Projects دسترسی دارد

Rule 2: Permission Checking

🟢 Verified — Multiple services

CanManageProjects: Workspace Owner یا Admin

CanManageSprints: Workspace Owner یا Admin

CanManageTask: Project Owner یا Manager

CanManageProject: Workspace Owner/Admin یا Project Owner

Rule 3: Sprint Lifecycle

🟢 Verified — SprintService.cs

Planning → Active → Completed/Cancelled

فقط یک Sprint فعال در هر پروژه مجاز است

Active شدن → سایر Sprints فعال به Planning برمی‌گردند

Rule 4: Dependency Cycle Prevention

🟢 Verified — TaskDependencyService.WouldCreateCycleAsync

قبل از اضافه کردن وابستگی، BFS انجام می‌شود

اگر TaskA به TaskB وابسته باشد و TaskB قبلاً به TaskA وابسته باشد → Reject → Cycle detected

Rule 5: Overdue Cascade

🟢 Verified — OverdueCascadeBackgroundService

هر ۳۰ دقیقه:

بررسی Task های عقب‌افتاده

اگر هیچ assignee ای AutoCascade را فعال داشته باشد

→ محاسبه روزهای تأخیر

→ بررسی زنجیره وابستگی

→ به‌روزرسانی موعد Task های وابسته

→ ثبت OverdueCascadeLog

→ ارسال اعلان به assignee ها

Rule 6: Chat Rate Limiting

🟢 Verified — ChatService.cs

حداکثر ۵ پیام در ۳ ثانیه

حداکثر طول محتوا: ۴۰۰۰ کاراکتر

فقط اعضای پروژه می‌توانند پیام ارسال کنند

Rule 7: Notification Preferences

🟢 Verified — NotificationService.IsNotificationEnabledAsync

System و Reminder همیشه فعال

سایر انواع بر اساس UserNotificationPreference

Rule 8: Chat Message Deletion

🟢 Verified — ChatService.DeleteMessageAsync

فرستنده پیام OR Owner/Manager پروژه می‌تواند حذف کند

حذف نرم (ViewState = false)

حذف فایل پیوست از فایل‌سیستم

Rule 9: Pin Message Permission

🟢 Verified — ChatService.TogglePinAsync

فقط Owner/Manager پروژه می‌تواند پیام را pin کند

Rule 10: Soft Delete Pattern

🟢 Verified — BaseEntity + Query Filters

ViewState = true: رکورد فعال

ViewState = false: حذف نرم

Query Filters در DbContext → خودکار فیلتر شدن رکوردهای حذف‌شده

---

15. Features Inventory

Authentication

🟢 Verified — AccountController.cs

Feature	Status	Evidence
Register	Fully Implemented	AccountController.Register

Login	Fully Implemented	AccountController.Login
Logout	Fully Implemented	AccountController.Logout
Password Reset	Fully Implemented	ForgotPassword/ResetPassword
Email Confirmation	Fully Implemented	ConfirmEmail
Google OAuth	Partially Implemented	Code exists but commented out in Program.cs
Session Tracking	Fully Implemented	UserSessionTracker

Workspace Management

🟢 Verified

Feature	Status
Create/Edit/Delete	Fully Implemented
Member Management	Fully Implemented
Invitation System	Fully Implemented
Settings	Fully Implemented
Dashboard	Fully Implemented

Project Management

🟢 Verified

Feature	Status
Create/Edit/Archive	Fully Implemented
Member Management	Fully Implemented
Team Assignment	Fully Implemented
Settings	Fully Implemented
Dashboard	Fully Implemented
Labels	Fully Implemented

Sprint / Agile

Verified

Feature	Status
Sprint CRUD	Fully Implemented
Sprint Activation/Completion	Fully Implemented
UserStory Management	Fully Implemented
Product Backlog	Fully Implemented
Burndown Chart	Fully Implemented
Velocity Chart	Fully Implemented
Sprint Report (AI)	Fully Implemented
Planning View	Fully Implemented

Task Management

Verified

Feature	Status
Task CRUD	Fully Implemented
Task Board (Kanban)	Fully Implemented
Task Assignment	Fully Implemented
Status Workflow	Fully Implemented
Priority	Fully Implemented
Task Types	Fully Implemented
Sub-tasks	Fully Implemented
Checklists	Fully Implemented
Comments	Fully Implemented

Attachments	Fully Implemented
Labels	Fully Implemented
Time Tracking	Fully Implemented
Reminders	Fully Implemented
Task Trade/Exchange	Fully Implemented
Quick Add	Fully Implemented

Dependency Management

Verified

Feature	Status
Add/Remove Dependencies	Fully Implemented
Cycle Detection	Fully Implemented
Dependency Graph	Fully Implemented
Impact Analysis	Fully Implemented
Risk Overview	Fully Implemented
Overdue Cascade	Fully Implemented

Smart Features (Algorithms)

Verified

Feature	Status
Priority Engine	Fully Implemented
Delay Risk Analysis	Fully Implemented
Project Health Score	Fully Implemented
Workload Analysis	Fully Implemented
AI Task Breakdown	Fully Implemented

AI Sprint Reports	Fully Implemented
AI Risk Narrative	Fully Implemented

Communication

Verified

Feature	Status
Group Chat (per project)	Fully Implemented
Message Threads	Fully Implemented
Reactions	Fully Implemented
Pin Messages	Fully Implemented
Typing Indicators	Fully Implemented
Online/Offline Presence	Fully Implemented
Read State	Fully Implemented
Search	Fully Implemented
Push Notifications	Fully Implemented
File Attachments	Fully Implemented

Reports

Verified

Feature	Status
Workspace Report	Fully Implemented
Project Report	Fully Implemented
PDF Export	Fully Implemented
Excel Export	Fully Implemented
Agile Dashboard	Fully Implemented

Notifications

🟢 Verified

Feature	Status
Real-time (SignalR)	Fully Implemented
Notification Preferences	Fully Implemented
Mark Read/Unread	Fully Implemented
Push Notifications (Webpushr)	Fully Implemented
8 Notification Types	Fully Implemented

Settings

🟢 Verified

Feature	Status
Account Settings	Fully Implemented
Notification Preferences	Fully Implemented
Appearance (Theme)	Fully Implemented
Management Settings	Fully Implemented
Security (Sessions)	Fully Implemented
Logout All Devices	Fully Implemented
Delete Account	Fully Implemented

Offroad Tasks

🟢 Verified — OffroadController, OffroadTask entity

Feature	Status
Offroad Task CRUD	Fully Implemented
Out-of-scope work tracking	Fully Implemented

---

16. Views (30 View Directories)

🟢 Verified — Views/ directory listing

View Directory	Controller	Purpose
Account	AccountController	Login, Register, Profile, Password
Home	HomeController	User Dashboard
Workspace	WorkspaceController	Workspace list, details
WorkspaceDashboard	WorkspaceDashboardController	Workspace dashboard
WorkspaceMember	WorkspaceMemberController	Member management
WorkspaceReport	WorkspaceReportController	Workspace reports
Project	ProjectController	Project CRUD
ProjectDashboard	ProjectDashboardController	Project dashboard
ProjectMember	ProjectMemberController	Project members
ProjectReport	ProjectReportController	Project reports
Sprint	SprintController	Sprint management
Task	TaskController	Task details/edit
TaskBoard	TaskBoardController	Kanban board
Backlog	BacklogController	Product backlog
Dependency	DependencyController	Dependency management
DelayRisk	DelayRiskController	Risk analysis
Workload	WorkloadController	Workload analysis
Chat	ChatController	Group chat
Label	LabelController	Label management

Notification	NotificationController	Notification center
Settings	SettingsController	User settings
Admin	AdminController	Admin dashboard
Team	TeamController	Team management
TaskTrade	TaskTradeController	Task exchange
Offroad	OffroadController	Offroad tasks
UserStory	UserStoryController	Story management
Reminder	ReminderController	Reminder management
Activity	ActivityController	Activity logs
Report	ReportController	Global reports
AgileDashboard	AgileDashboardController	Agile metrics
Shared	-	Layout, Sidebar, Modals, Components

---

17. JavaScript (33 Files)

🟢 Verified — wwwroot/js/

File	Purpose	Key Functions
site.js	Global utilities	SweetAlert, theme, formatting
notification.js	Notification bell	Load, mark read, real-time updates
chat.js	Group chat	SignalR messaging, reactions, search
task.js	Task details	Edit, status, comments
taskboard.js	Kanban board	Drag-drop, status changes
task-quick-add.js	Quick add modal	Fast task creation

backlog.js	Product backlog	Story management, drag-drop
planning.js	Sprint planning	Story assignment
sprint.js	Sprint management	CRUD, activate/complete
dependency.js	Dependency management	Add/remove dependencies
dependency-graph.js	Dependency visualization	Graph rendering (vis.js/network)
delayrisk.js	Delay risk UI	Risk overview, AI narrative
workload.js	Workload analysis	Capacity, utilization charts
project.js	Project management	CRUD, settings
project-dashboard.js	Project dashboard	Charts, stats
project-settings.js	Project settings	Preferences
workspace-dashboard.js	Workspace dashboard	Charts, stats
workspace-member.js	Workspace members	Invite modal, role management
workspace-settings.js	Workspace settings	Preferences
settings.js	User settings	All settings tabs, sessions, delete
profile.js	User profile	Avatar, info
admin-dashboard.js	Admin dashboard	System stats
agile-dashboard.js	Agile metrics	Charts
label.js	Label management	CRUD
offroad.js	Offroad tasks	CRUD
tasktrade.js	Task exchange	Request/response
userstory.js	User story	CRUD
reminder.js	Reminders	CRUD

home-dashboard.js	Home dashboard	Personal stats
GlobalSearch.js	Global search	Search bar
date-picker-init.js	Date picker	Jalali/Gregorian
delayrisk.js	Risk analysis	API calls

---

18. CSS/UI (41 Files)

🟢 Verified — wwwroot/css/

Design System

- **Colors**: Primary #5B5FEF, Grays gray100-gray700, Status colors (Green #22C55E, Purple #8B5CF6, Red #EF4444)
- **Typography**: Vazirmatn (Persian), system font stack
- **Layout**: RTL-first, responsive
- **Cards**: 12px border-radius, hover lift
- **Buttons**: Primary .workspace-primary-btn, secondary .workspace-filter
- **Tabs**: Bottom border indicator with slide animation

CSS Organization

File	Purpose
layout.css	Global layout, grid, containers
site.css	Global styles, typography, colors
auth.css	Login/Register pages
workspace.css	Workspace pages
workspace-pages.css	Shared workspace components
workspace-index.css	Workspace list
workspace-dashboard.css	Workspace dashboard

workspace-settings.css	Workspace settings
workspace-member.css	Member management
project.css	Project pages
project-dashboard.css	Project dashboard
project-settings.css	Project settings
sprint.css	Sprint management
task.css	Task pages
taskboard.css	Kanban board
task-pages.css	Task shared styles
backlog.css	Backlog
planning.css	Sprint planning
dependency.css	Dependency management
delayrisk.css	Risk analysis
workload.css	Workload analysis
chat.css	Group chat
chat-reactions.css	Chat reactions
notification.css	Notifications
settings.css	User settings
profile.css	Profile page
admin-dashboard.css	Admin dashboard
agile-dashboard.css	Agile metrics
report.css	Reports
sprintreport.css	Sprint reports

label.css	Labels
offroad.css	Offroad tasks
tasktrade.css	Task trade
team.css	Team management
userstory.css	User stories
reminder.css	Reminders
activity.css	Activity logs
custom-scrollbar.css	Custom scrollbar
custom-select.css	Custom select dropdowns
change-password.css	Password change

---

19. Authentication & Authorization

Authentication

🟢 Verified — Program.cs, AccountController.cs

- **ASP.NET Core Identity** with ApplicationUser (int primary key)
- **Cookie Authentication** (SmartTask cookie, sliding expiration, 7-day expiry)
- **Google OAuth** (code exists, commented out in Program.cs)
- **Email Confirmation** required before login
- **Session Tracking** via UserSession entity

Authorization

🟢 Verified — Program.cs

- **Role-based**: Admin, ProjectManager, Member
- **Policies**: AdminOnly, ProjectManagerOnly, MemberOnly
- **Resource-level**: CanManageTask, CanManageProject, CanManageSprints etc.

- **[Authorize]** attribute on controllers/actions
- **[ValidateAntiForgeryToken]** on all POST actions

Session Security

🟢 Verified — UserSessionTracker.cs

- Device/browser/OS detection from User-Agent
- Session token per login
- Logout All Devices via SecurityStamp update
- Session revocation on logout

---

20. Notification System

🟢 Verified — NotificationService.cs, NotificationHub.cs

- **8 Types**: System, Reminder, Assignment, Comment, Mention, Invitation, StatusChange, Deadline
- **Real-time**: SignalR NotificationHub pushes to user-specific groups
- **Preferences**: Per-type enable/disable via UserNotificationPreference
- **Push**: Webpushr for browser push notifications
- **Batch Operations**: Create, mark-read, delete in batch

---

21. Chat / Communication

🟢 Verified — ChatService.cs, ChatHub.cs

- **Per-project group chat** (one chat room per project)
- **SignalR** for real-time messaging
- **Features**: Send, Edit, Delete, Reply (threads), Reactions (emoji), Pin, Typing indicators
- **Presence**: Online/Offline tracking via PresenceTracker (Singleton, in-memory)
- **Read State**: ChatReadState tracks last read message per user per project

- **Mention**: @username pattern with notification
- **Search**: Full-text search in messages
- **Rate Limiting**: 5 messages per 3 seconds
- **Push Notifications**: Webpushr for offline members

---

22. Agile / Scrum Features

🟢 Verified

- **Product Backlog**: One per project, holds UserStories
- **User Stories**: With Story Points, Business Value, Priority, Status, Acceptance Criteria
- **Sprints**: Planning → Active → Completed/Cancelled lifecycle
- **Sprint Planning**: Drag stories from backlog to sprint
- **Task Board**: Kanban with ToDo/InProgress/InReview/Done/Cancelled columns
- **Burndown Chart**: Ideal vs. actual remaining points
- **Velocity Chart**: Planned vs. completed points across sprints
- **Capacity Management**: Weekly capacity hours per member

---

23. Algorithms & Metrics

Algorithm 1: Priority Engine

🟢 Verified — PriorityEngineService.cs

- **Input**: Task with due date, dependencies, assignees
- **Formula**: TotalScore = UrgencyScore(0-40) + DependencyScore(0-35) + WorkloadScore(0-25)
- **Output**: Suggested priority level (Lowest to Highest)
- **Purpose**: Data-driven prioritization replacing subjective decisions

Algorithm 2: Delay Risk Analysis

🟢 Verified — DelayRiskService.cs

- **Input**: Project tasks, members, dependencies, cascade logs
- **Formula**: RiskScore = OverdueScore(0-40) + WorkloadScore(0-30) + DependencyScore(0-20) + CascadeScore(0-10)
- **Output**: Risk level (low/medium/high/critical) with breakdown
- **Purpose**: Quantitative risk assessment

Algorithm 3: Project Health Score

🟢 Verified — ProjectHealthService.cs

- **Input**: Risk data from DelayRiskService, task completion counts
- **Formula**: HealthScore = Schedule(0.30) + Workload(0.25) + Dependency(0.20) + Delivery(0.25)
- **Output**: Health level (excellent/good/fair/poor) with sub-scores
- **Purpose**: Holistic project health assessment

Algorithm 4: Dependency Cycle Detection

🟢 Verified — TaskDependencyService.WouldCreateCycleAsync

- **Algorithm**: BFS traversal of dependency graph
- **Purpose**: Prevent circular dependencies

Algorithm 5: Dependency Impact Analysis

🟢 Verified — TaskDependencyService.GetImpactedTasksAsync

- **Algorithm**: BFS traversal following dependent tasks
- **Output**: List of impacted tasks with depth and chain requirement flag
- **Purpose**: Show cascade impact of a task delay

Algorithm 6: Overdue Cascade Engine

🟢 Verified — OverdueCascadeBackgroundService

- **Schedule**: Every 30 minutes
- **Logic**: Detect overdue → follow dependency chains → adjust due dates → log → notify

- **User Setting**: Respects AutoCascadeDependencyDates per user

Algorithm 7: Workload Utilization

🟢 Verified — WorkloadAnalysisService

- **Formula**: Utilization = (AssignedHours / CapacityHours) × 100
- **Levels**: under (<80%), balanced (80-100%), overloaded (>100%)

Metrics/KPIs

🟢 Verified

Metric	Source	Formula
Priority Score	PriorityEngineService	3-factor weighted sum
Risk Score	DelayRiskService	4-factor weighted sum
Health Score	ProjectHealthService	4-dimension weighted average
Utilization %	WorkloadAnalysisService	assigned/capacity
Completion Rate	SprintService	completedPoints/totalPoints
Burndown	SprintService	Ideal vs. actual remaining
Velocity	SprintService	Planned vs. completed points

---

24. Workflows

Workflow: User Registration

User → Register View → AccountController.Register →

UserManager.CreateAsync → Email Confirmation Token →

SendEmail (SMTP) → User clicks link → ConfirmEmail → Login

Workflow: Create Project

Workspace Member → Project/Create → ProjectController.Create →

ProjectService.AddAsync → DbContext → Database →

ActivityLog → Notification to workspace members

Workflow: Create Task

Project Member → Task Create → TaskController →

TaskService.AddAsync → DbContext → Database →

Notification to assignees → ActivityLog

Workflow: Task Dependency Cascade

Background Service (30min) →

Find overdue tasks → Check AutoCascadeDependencyDates →

BFS dependency chain → Calculate delay days →

Update impacted task due dates →

Create OverdueCascadeLog →

Send notifications → Log activity

Workflow: Real-time Chat

User types message → ChatHub.SendMessage →

ChatService.SendMessageAsync → Database →

SignalR broadcast to project group →

Webpushr push to offline members →

Mention notifications → Read state update

Workflow: Sprint Report (AI)

Scrum Master clicks Generate → SprintReportAiService.GenerateReportAsync →

Collect sprint stats → Build prompt →

AiClientService.GetCompletionAsync → LM Studio →

Store SprintReport → Display to user

Workflow: Logout All Devices

Settings → Security → Logout All →

SettingsController.LogoutAllDevices →

UpdateSecurityStampAsync (invalidates all cookies) →

Revoke all UserSessions →

Re-sign in current user →

Show success toast

---

25. Configuration

🟢 Verified — appsettings.json

Config	Purpose	Sensitive
ConnectionStrings.DefaultConnection	SQL Server	Secret detected — value intentionally omitted
EmailSettings	SMTP Gmail	Secret detected — value intentionally omitted
OpenAI/LmStudio	LM Studio API	Base URL + model
Webpushr	Push notification service	Secret detected — value intentionally omitted
App	Application URL	No
Authentication	Google OAuth (commented out)	Secret detected — value intentionally omitted

---

26. Performance Observations

🟡 Inferred from code analysis

Positive Patterns

- **OPTIMIZED comments** throughout services indicating performance awareness
- **Server-side counting** instead of loading all records (CountAsync vs ToListAsync)
- **Pre-computed maps** in WorkloadAnalysisService to avoid O(n^2)
- **Batch operations** in NotificationService
- **In-memory graph traversal** in TaskDependencyService instead of N+1 DB queries

- **Filtered indexes** on frequently queried columns
- **Query filters** for soft delete

Potential Issues

- **Lazy loading risk**: Some Include() chains could be missed
- **PresenceTracker**: In-memory dictionary (Singleton) — not distributed-safe
- **ChatService**: ConcurrentDictionary for rate limiting — not distributed
- **OverdueCascadeBackgroundService**: Loads all overdue tasks into memory
- **N+1 in UserSessionTracker.GetActiveSessionsAsync**: No Include for ApplicationUser

---

27. Security Observations

🟢 Verified from code analysis

- ✅ **CSRF Protection**: [ValidateAntiForgeryToken] on all POST actions
- ✅ **Soft Delete**: ViewState pattern prevents data loss
- ✅ **Session Tracking**: Login sessions tracked with device info
- ✅ **SecurityStamp**: Used for logout-all invalidation
- ✅ **Password Policy**: Min 6 chars, digit required, unique emails
- ✅ **Input Validation**: ModelState validation on forms
- ✅ **Chat Rate Limiting**: Prevents spam
- ⚠️ **Admin Password**: Hardcoded in AdminSeeder.cs ("Admin123!")
- ⚠️ **No HTTPS enforcement** in Development
- ⚠️ **Console.WriteLine** used for logging instead of ILogger in some places

---

28. Dead Code & Unused Components

Commented-Out Code

🟢 Verified — Program.cs

- Google OAuth authentication is fully commented out

Excluded from Build

🟢 Verified — SmartTask.Web.csproj

- Middlewares/ folder: Excluded from compilation
- Repositories/ folder: Excluded from compilation (replaced by Infrastructure/Repositories/)

Potential Dead Code

🟡 Inferred

- ContextRequirementType enum: Search results suggest limited usage
- Components/SmartTaskSimple.jsx: React component that may be experimental

---

29. Incomplete / Partial Features

Google OAuth

🟠 Partial — AccountController.ExternalLogin exists but commented out in Program.cs

File Upload

🟡 Inferred — IFileUploadService and FileUploadService exist; Attachment entity exists; Chat file upload exists; but no dedicated upload controller endpoint for task attachments was found in the controllers I inspected

Dark Mode Theme

🟠 Partial — ThemeType enum has Dark and System values; Theme property exists on ApplicationUser; but CSS for dark theme was not found in the 41 CSS files

Time Logging

🟠 Partial — TimeLog entity exists; TimeLogService exists; TimeLogController exists; but integration with task views for start/stop timers was not confirmed

Task Trade

🟢 Verified — TaskTradeController, TaskTradeService, TaskTradeRequest entity — appears fully implemented

---

30. Strengths

1. **Service Layer Architecture**: Clean separation between Controller → Service → Repository → DbContext
2. **Generic Repository + Unit of Work**: Reusable data access pattern
3. **Comprehensive Business Logic**: Priority engine, risk analysis, health scoring — genuine algorithmic value beyond CRUD
4. **Background Services**: Automated cascade and reminder processing
5. **Real-time Communication**: SignalR for chat and notifications
6. **AI Integration**: LLM-powered sprint reports and task breakdown
7. **Soft Delete**: Consistent ViewState pattern with query filters
8. **Performance Optimization**: Evidence of OPTIMIZED patterns throughout
9. **Rich Entity Model**: 34 entities covering the full domain
10. **Persian/RTL Native**: Full RTL support with Jalali date support

---

31. Weaknesses

1. **In-Memory Presence**: PresenceTracker is Singleton dictionary — not distributed-safe for scaling
2. **Hardcoded Credentials**: AdminSeeder password, email settings in appsettings.json
3. **Mixed Patterns**: Some services use Repository + UnitOfWork, others inject DbContext directly
4. **Large Service Layer**: 46 services in flat structure — no domain grouping
5. **No Caching**: No Redis or distributed caching observed
6. **No API Layer**: All endpoints are MVC views — no REST API for mobile/third-party

7. **No Logging Framework**: Console.WriteLine used instead of ILogger in several places
8. **Test Coverage**: Test project exists but only 8 test files included
9. **Google OAuth**: Code exists but disabled — incomplete feature
10. **No Input Sanitization**: Chat content not sanitized for XSS (though SignalR + Razor encoding helps)

---

32. Innovation & Differentiators

1. Multi-Factor Priority Engine

🟢 Verified — PriorityEngineService.cs
Combines time urgency, dependency impact, and workload pressure into a single score. This is **not** available in simple task managers like Trello or Asana.

2. Quantitative Delay Risk Analysis

🟢 Verified — DelayRiskService.cs
Calculates a composite risk score from 4 factors with specific weights. Goes beyond simple overdue counts.

3. Composite Health Score

🟢 Verified — ProjectHealthService.cs
4-dimension weighted health assessment. Novel approach for project management tools.

4. Automatic Dependency Cascade

🟢 Verified — OverdueCascadeBackgroundService.cs
Background service that automatically propagates delays through dependency chains. With user-level opt-out control.

5. AI-Powered Sprint Reports

🟢 Verified — SprintReportAiService.cs
LLM-generated sprint retrospectives with structured data input.

6. AI Task Breakdown

🟢 Verified — TaskBreakdownService.cs
Automatic decomposition of tasks into sub-tasks using LLM.

---

33. Comparison with Simple Task Managers

Simple Task Manager (e.g., Trello)

↓

Task CRUD

Board columns

Basic labels

SmartTask

↓

Hierarchical Organization (Workspace → Project → Sprint → Story → Task)

↓

Agile Workflow (Sprint Planning, Backlog, Burndown, Velocity)

↓

Smart Priority Engine (3-factor scoring)

↓

Delay Risk Analysis (4-factor risk scoring)

↓

Project Health Scoring (4-dimension weighted)

↓

Automatic Dependency Cascade (background service)

↓

Real-time Communication (chat, notifications)

↓

AI-Powered Reports & Task Breakdown

↓

Workload Analysis & Capacity Management

↓

Dependency Graph Visualization & Cycle Detection

---

34. Proposal Mapping

Proposal Section (پروپوزال)	Project Component
بیان مسئله	Problem Domain
اهداف پروژه	Features
نوآوری / الگوریتم	Priority Engine
نوآوری / تحلیل ریسک	Risk Analysis
نوآوری / معیارهای هوشمند	Health Score
نوآوری / مدیریت خودکار	Auto Cascade
معماری سیستم	Architecture
طراحی پایگاه داده	Entity Model + ERD
فناوری‌های مورد استفاده	Technology Stack
روش تحقیق / الگوریتم‌ها	Algorithms
معیارهای ارزیابی	Metrics
پیاده‌سازی رابط کاربری	UI/UX
ملاحظات امنیتی	Security
بهینه‌سازی	Performance

---

35. Recommended Proposal Title

ساده

1. طراحی و پیاده‌سازی سامانه مدیریت پروژه چابک با قابلیت‌های هوشمند

دانشگاهی

2. طراحی و پیاده‌سازی سامانه مدیریت پروژه‌های نرم‌افزاری با تکیه بر مفاهیم Agile و الگوریتم‌های هوشمند اولویت‌بندی و تحلیل ریسک

تخصصی و قوی

3. :SmartTaskطراحی، پیاده‌سازی و ارزیابی یک سامانه مدیریت پروژه چابک مبتنی بر ASP.NET Core با موتور اولویت‌بnehindی چندعامله، تحلیل کمّی ریسک تأخیر و پشتیبانی از یادگیری ماشین برای گزارش‌سازی اسپرینت

بهترین عنوان :شماره ۲ — کامل، دقیق، و قابل دفاع در جلسه دفاع.

---

36. Self-Audit Checklist

- تمام Folders — بررسی شد (Controllers, Services, Models, Data, Infrastructure, Hubs, Views, wwwroot, Common, Components, ViewComponents, Migrations)
- تمام Controllers — 41 controller بررسی شد
- تمام Services — 46 service interface + implementation بررسی شد
- تمام Entities — 34 entity بررسی شد
- تمام Enums — 23 enum بررسی شد
- تمام ViewModels — 30+ subdirectory بررسی شد
- تمام Views — 30 view directory بررسی شد
- تمام JavaScript — 33 فایل فهرست شد
- تمام CSS — 41 فایل فهرست شد
- تمام Migrations — 20 migration بررسی شد
- DbContext — بررسی شد
- EF Configurations — 34 configuration بررسی شد
- Authentication — بررسی شد
- Authorization — بررسی شد
- Notifications — بررسی شد
- Chat — بررسی شد

- 6 — Algorithmsالگوریتم استخراج شد
- — Configurationبررسی شد
- — Seed Dataبررسی شد
- — Background Servicesبررسی شد
- — Hubsبررسی شد
- — ViewComponentsبررسی شد
- — Test Projectبررسی شد
- — Package Referencesبررسی شد
- — Program.csبررسی شد

Final System Definition

SmartTaskیک سامانه مدیریت پروژه چابک (Agile Project Management) مبتنی بر ASP.NET Core 8.0 MVCبا پایگاه داده SQL Server است که برای تیم‌های توسعه نرم‌افزار فارسی‌زبان طراحی شده است. این سامانه سلسله‌مراتب سازمانی **Workspace → Project → Sprint → UserStory → Task**را پیاده‌سازی می‌کند و ویژگی‌های هوشمندی فراتر از ابزارهای ساده مدیریت وظایف ارائه می‌دهد:

الگوریتم‌های هوشمند :موتور اولویت‌بندی چندعامله(Urgency 40% + Dependency 35% + Workload 25%) ، تحلیل کمّی ریسک تأخیر(Overdue 40% + Workload 30% + Dependency 20% + Cascade 10%) ، و امتیاز سلامت پروژه ترکیبی (Schedule 30% + Workload 25% + Dependency 20% + Delivery 25%) —این سه الگوریتم مکمل یکدیگرند و برای اولین بار در یک سامانه مدیریت پروژه فارسی ادغام شده‌اند.

مدیریت خودکار وابستگی‌ها :یک Background Service هر ۳۰ دقیقه زنجیره وابستگی تسک‌ها را بررسی و در صورت تأخیر، موعد تسک‌های وابسته را به‌صورت خودکار به‌روزرسانی می‌کند — قابلیتی که در اکثر ابزارهای موجود وجود ندارد.

ارتباطات بلادرنگ :چت گروهی هر پروژه باSignalR ، واکنش‌های ایموجی، پین پیام، وضعیت آنلاین/آفلاین، و اعلان‌های Push از طریق.Webpushr

هوش مصنوعی: تولید گزارش‌های اسپرینت و شکستن خودکار تسک‌ها با استفاده از مدل‌های زبانی.(LM Studio)

معماریMVC + Service Layer + Generic Repository + Unit of Work :، با Entity34 ، Service46 ، Controller41 ، Migration20 ، و یک مدل داده غنی شامل 34 جدول.